Admin Panel — CIE-1 Study Guide

> Read this top-to-bottom before your exam. Every section explains **what the file does**, **how the code works**, and **what to say when asked**.

TABLE OF CONTENTS

1. Project Overview — What is This Website?
2. Folder Structure of the Admin Panel
3. Core Infrastructure Files
    - includes/init.php
    - includes/config.php
    - includes/db.php
    - includes/auth.php
    - Common/header.php
4. Admin Pages — File by File
    - user_management.php
    - add_user.php
    - provision_temp_user.php
    - project_management.php
    - payment_gateway.php
    - leave_management.php
    - file_viewer.php
5. Design System — Colors, Fonts, Tailwind
6. Form Validation — How It Works
7. Authentication & Role Guard — How Login Protection Works
8. Database Connection Pattern
9. Common Questions Faculty May Ask + Answers
10. Quick-Reference Code Snippets

1. Project Overview

Website Name: Ripal Design
Type: Architecture / Interior Design firm website
Purpose: A multi-role web application for an architecture firm to manage projects, workers, clients, and finances.

Three types of users:

Role	What they can do
Admin	Full control — manage users, projects, payments, leave
Worker	View assigned projects, submit ratings
Client	Upload drawings, view project progress, submit revisions

Your responsibility (Member 3 — Admin):

- Design the admin dashboard
- Design all admin pages with proper UI
- Add form validation on all admin forms
- Ensure admin-only pages redirect non-admins away

2. Folder Structure

```
admin/
  add_user.php          ← Form to create a new user account
  provision_temp_user.php ← Generate temporary credentials for a worker/employee
  user_management.php   ← List, view, edit, delete all users
  project_management.php← View and create projects (project cards grid)
  payment_gateway.php   ← Financial transactions ledger
  leave_management.php  ← Approve/reject employee leave requests
  file_viewer.php       ← View project PDF/blueprint files

includes/
  init.php      ← MASTER bootstrap — loads everything
  config.php    ← BASE_URL, BASE_PATH constants
  db.php        ← Database (PDO) connection
  auth.php      ← Login check and role check functions

Common/
  header.php    ← HTML <head> assets + navigation bar (used by admin pages)
  header_alt.php← Alternate nav header (sidebar slide-out)
  footer.php    ← Footer included at the bottom of pages
```

3. Core Infrastructure Files

3.1 `includes/init.php`

What it does: This is the FIRST line in almost every admin page. It loads all other core files in the correct order.

```php
require_once __DIR__ . '/../includes/init.php';
```

Loading order inside init.php:

1. `config.php` → sets up `BASE_URL`, `BASE_PATH`, `PROJECT_ROOT` constants
2. `db.php` → creates `$pdo` (database connection)
3. `auth.php` → loads helper functions like `require_role()`, `is_logged_in()`
4. `util.php` → utility/helper functions

5. Starts PHP session with `session_start()`

Why use one bootstrap file? So you don't have to write 4 separate `require_once` calls on every page — just one line does everything.

3.2 `includes/config.php`

What it does: Detects where the application is installed and defines constants for building URLs.

Key constants:

- `BASE_URL` → e.g., `http://localhost/TheFinal_Thefinal2`
- `BASE_PATH` → e.g., `/TheFinal_Thefinal2`
- `PROJECT_ROOT` → Absolute filesystem path

How it auto-detects the path:

```
// It checks the current script's folder name
$parts = explode('/', $scriptPath);
$lastPart = $parts[count($parts) - 1];

// If we're inside admin/, dashboard/, public/, etc., it removes that folder
$appFolders = ['public', 'dashboard', 'admin', 'client', 'worker'];
if (in_array($lastPart, $appFolders)) {
    array_pop($parts); // removes the last folder to get root
}
```

Why this matters: Any link we write as `BASE_PATH . '/admin/user_management.php'` will work whether the site is at `localhost/mysite/` or at `localhost/` — the path adjusts automatically.

3.3 `includes/db.php`

What it does: Creates a **PDO** (PHP Data Objects) connection to the MySQL database.

PDO = the safe, modern way to talk to a database in PHP.

```
$dsn = "mysql:host=localhost;port=3306;dbname=ripal_db;charset=utf8mb4";
$pdo = new PDO($dsn, $DB_USER, $DB_PASS, $options);
```

Key PDO options used:

Option	What it means
`ERRMODE_EXCEPTION`	Throw exceptions on SQL errors (so we can catch them)
`DEFAULT_FETCH_MODE => FETCH_ASSOC`	Results come back as named arrays, not numbered
`EMULATE_PREPARES => false`	Use real prepared statements (prevents SQL injection)

Fallback / Demo Mode:
If the database is unavailable (connection fails), `$pdo` is set to `null`. Pages then use **hardcoded static data** so the UI still works for demo/CIE purposes.

```
// Example fallback in dashboard.php:
if (isset($pdo) && $pdo instanceof PDO) {
    // Load from database
} else {
    // Use static demo data
    $projects = [['id' => 1, 'name' => 'Shanti Sadan'], ...];
}
```

Two helper functions:

```
db_connected(); // returns true/false
get_db();       // returns $pdo or null
```

3.4 `includes/auth.php`

What it does: Provides all authentication-related functions. This is how we protect admin pages.

Functions defined:

`require_login()`

Checks if `$_SESSION['user']` exists. If the user is not logged in → **redirects to login page and exits**.

`require_role('admin')`

First calls `require_login()`, then checks if the logged-in user's role matches. If not → **redirects to homepage**.

```
// HOW ADMIN PAGES ARE PROTECTED:
require_once __DIR__ . '/../includes/init.php';
require_role('admin');   // ← This one line protects the entire page
```

`current_user()`

Returns the `$_SESSION['user']` array (has username, role, etc.)

`is_logged_in()`

Returns `true` or `false`.

`has_role('admin')`

Returns `true` if the logged-in user has that specific role.

How session-based auth works:

1. User logs in via `public/login.php`
2. On successful login, `$_SESSION['user']` is set with the user's data
3. Every protected page calls `require_role('admin')` which checks that session variable
4. If session is empty or role doesn't match → user is kicked to login or home

3.5 `Common/header.php`

What it does: Loads all CSS/JS libraries for every page and renders the navigation bar.

Libraries it loads:

- **Google Fonts** — Inter (body text) + Playfair Display (headings)
- **Bootstrap 5.3.2** — Grid, utilities
- **Bootstrap Icons** — `bi-*` icon classes
- **Lucide Icons** — `data-lucide="icon-name"` SVG icons (modern, clean)
- **Tailwind CSS** (CDN) — Utility-first CSS framework

Tailwind custom theme configured here:

```
tailwind.config = {
  theme: {
    extend: {
      colors: {
        "rajkot-rust": "#94180C",    // Primary brand color (dark red)
        "canvas-white": "#F9FAFB",   // Page background
        "foundation-grey": "#2D2D2D",// Dark text/header color
        "approval-green": "#15803D", // Success states
        "pending-amber": "#B45309",  // Warning states
        "slate-accent": "#334155",   // Workers/secondary
      }
    }
  }
}
```

Navigation:
The nav is a **sidebar slide-out** (hamburger menu). Links are grouped into:

- Dashboard (Dashboard Home, Profile, Review Requests)
- Worker Portal (Worker Dashboard, Assigned Projects, Ratings)
- Administration (Project Portfolio, User Controls, Leave Manager, Financial Gateway)

How to include it in a page:

```
<?php $HEADER_MODE = 'dashboard'; require_once __DIR__ . '/../Common/header.php'; ?>
```

Setting `$HEADER_MODE = 'dashboard'` also loads `admin-responsive.css` for admin-specific mobile styles.

4. Admin Pages

4.1 `user_management.php`

Purpose: Shows all registered users in a table. Admin can view, edit, or delete any user.

What the page contains:

Stats Cards (Top Row)

Four metric cards showing:

- **Total Registry** — 124 total users
- **Active Clients** — 42
- **Field Workers** — 68
- **System Staff** — 14

These are currently static/hardcoded values meant to demonstrate the UI layout.

Toolbar

- **Search input** — "Filter identities..." (client-side filter, visual only in current version)
- **Role dropdown** — "All Permissions", Administrators, Design Lead, etc.
- **Apply button** — triggers filter

User Table

Columns: Identity Profile | Authorization Level | Signal (Online Status) | Last Sync | Actions

Each row has three action buttons:

1. **Eye icon** → `href="../dashboard/profile.php?user=..."` — View user profile
2. **Settings icon** → `href="add_user.php?id=..."` — Edit user
3. **Trash icon** → `onclick="confirmDelete('Name')"` — Delete with confirmation

Mobile responsive table:
On mobile, each table row becomes a card using CSS:

```
/* In admin-responsive.css */
/* Rows become stacked blocks on small screens */
tr { display: block; }
td::before { content: attr(data-label); } /* Shows column label */
```

This works because each `<td>` has a `data-label="Column Name"` attribute.

Delete confirmation:

```
function confirmDelete(userName) {
    if (confirm('Are you sure you want to remove ' + userName + '?')) {
        // In full implementation: AJAX call or form submission
    }
}
```

Pagination placeholder:
A "Initialize Full Registry Scroll" button at the bottom (currently just a visual element, pagination not implemented yet — that's CIE-2).

4.2 `add_user.php`

Purpose: A form for the admin to create a new user account. After successful creation, redirects to `user_management.php`.

PHP Processing (Server-side):

```
require_once __DIR__ . '/../includes/init.php';
require_role('admin');  // Only admins can access
```

POST form handling:

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $firstName = trim($_POST['firstName'] ?? '');
    $email     = trim($_POST['email'] ?? '');
    $password  = $_POST['password'] ?? '';
    $role      = $_POST['role'] ?? 'client';

    // Validation
    if (empty($firstName) || empty($email) || empty($password)) {
        $error = 'All fields are required.';
    } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $error = 'Please enter a valid email address.';
    } else {
        // Check if email already exists
        $stmt = $db->prepare("SELECT id FROM users WHERE username = ?");
        $stmt->execute([$email]);
        if ($stmt->fetch()) {
            $error = 'A user with this email already exists.';
        } else {
            // Hash the password BEFORE saving (security best practice)
            $passwordHash = password_hash($password, PASSWORD_DEFAULT);

            // Insert the new user
            $stmt = $db->prepare("INSERT INTO users (username, password_hash, role) VALUES (?, 
            $stmt->execute([$email, $passwordHash, $role]);

            $success = 'User created successfully!';
            header("Refresh: 2; url=user_management.php"); // Redirect after 2 seconds
        }
    }
}
```

Validations performed:

Validation	How
All fields required	`empty()` check
Valid email format	`filter_var($email, FILTER_VALIDATE_EMAIL)`
Duplicate email check	`SELECT` query before insert
Password minimum length	`minlength="8"` on HTML input + backend empty check

Security: Why `password_hash()` ?

- Never store plaintext passwords
- `password_hash($password, PASSWORD_DEFAULT)` uses bcrypt automatically
- Stored hash looks like: `$2y$10$...` (cannot be reversed)
- On login, `password_verify($input, $hash)` is used to check

Form fields:

- First Name, Last Name (side by side grid)
- Email Address
- Password (min 8 chars)
- Role select: Client / Worker / Employee / Administrator

Submit button behavior (JS):

```
document.getElementById('addUserForm').addEventListener('submit', function(e) {
    const btn = this.querySelector('button[type="submit"]');
    btn.innerHTML = 'Initializing Account...'; // Shows loading state
    btn.disabled = true; // Prevents double submission
});
```

4.3 `provision_temp_user.php`

Purpose: More advanced user creation — generates a **temporary random password** for workers or employees and shows the credentials on screen so the admin can share them.

Special Feature — Demo Mode:

```
$DEMO_MODE = (isset($_GET['demo']) && $_GET['demo'] === '1');
if (!$DEMO_MODE) {
    require_role('admin'); // Skip auth check if ?demo=1 in URL
}
```

This allows the UI to be shown in demo without being logged in as admin (useful for CIE presentation).

Temporary Password Generation:

```php
$tempPassword = substr(bin2hex(random_bytes(4)), 0, 8);
// random_bytes(4) = 4 cryptographically random bytes
// bin2hex converts to hex string = 8 characters
// Example output: "a3f9bc12"
```

Falls back to `'tmp' . rand(1000, 9999)` if `random_bytes` fails.

Dynamic Sub-role Selection (jQuery + JavaScript):
When the admin selects a role, a second dropdown appears dynamically:

```javascript
const options = {
    worker:   ['Master Carpenter', 'Field Site Engineer', 'Electrical Specialist',
               'Plumbing Specialist', 'Masonry Craftsman', 'Other'],
    employee: ['Principal Structural Engineer', 'Design Principal',
               'Valuation Specialist', 'Strategic Planner', 'Other']
};

roleSelect.on('change', function() {
    const v = $(this).val();
    if (v === 'worker' || v === 'employee') {
        // Populate the sub-role dropdown dynamically
        options[v].forEach(function(o) {
            subroleSelect.append($('<option>').val(o).text(o));
        });
        subroleContainer.removeClass('hidden'); // Show the sub-role div
    } else {
        subroleContainer.addClass('hidden'); // Hide it for admin/client
    }
});
```

If "Other" is selected from the sub-role dropdown → a free-text input appears.

jQuery Validation Plugin:

```
$("#tempProvisionForm").validate({
    rules: {
        firstName: "required",
        lastName: "required",
        email: { required: true, email: true },
        role: "required"
    },
    messages: {
        firstName: "First name is mandatory",
        email: "Valid email address required",
        role: "Please select a system role"
    }
});
```

This is the `jquery.validation` plugin — it highlights the input border red and shows error messages under each field without a page reload.

Result Display:
After form submission, the page shows a "Generation Result" card with:

- Full Name
- System Role + Sub-role
- Username (email)
- Temporary Password (with a copy-to-clipboard button)

```
function copyPassword() {
    const text = document.getElementById('tempPass').innerText;
    navigator.clipboard.writeText(text).then(() => {
        alert('Password copied to clipboard: ' + text);
    });
}
```

4.4 `project_management.php`

Purpose: Shows all projects as cards in a grid. Admin can filter by region/status, search, and add new projects via a modal popup.

Page structure:

1. **Filter Bar** — Region buttons (Rajkot, Jam Khambhalia, Global) + Status dropdown + Search
2. **Project Card Grid** — 3 columns on desktop, stacks on mobile
3. **"Add Project" Card** — Dashed border placeholder that opens a modal on click
4. **Modal Form** — "Initialize New Venture" with full project creation form

Project Card anatomy:

```html
<div class="project-card" data-region="Rajkot" data-status="Construction Ongoing">
  <!-- Image with overlay -->
  <div class="h-56 relative overflow-hidden">
    <img src="..." class="group-hover:scale-110 transition"> <!-- Zoom on hover -->
    <div class="absolute inset-0 bg-gradient-to-t from-black/80">
      <span class="bg-approval-green">Construction Phase</span>
      <h3 class="text-white">Rajkot Smart City Plaza</h3>
    </div>
  </div>
  <!-- Card body -->
  <div class="p-6">
    <p>Rajkot Infrastructure District</p>
    <!-- Progress bar -->
    <div class="w-full bg-gray-100 h-1.5">
      <div class="bg-rajkot-rust h-full" style="width: 72%"></div>
    </div>
    <span>72%</span>
    <a href="../dashboard/project_details.php?id=1">Open Record</a>
  </div>
</div>
```

Region Filter (JavaScript):

```javascript
function filterRegion(region) {
    document.querySelectorAll('.project-card').forEach(card => {
        if (region === 'Global' || card.dataset.region === region) {
            card.style.display = 'flex'; // Show
        } else {
            card.style.display = 'none'; // Hide
        }
    });
}
```

The `data-region` attribute on each card is compared to the selected filter. `dataset.region` reads the `data-region` attribute in JavaScript.

New Project Modal:

```
function openVentureModal() {
    document.getElementById('ventureModal').style.display = 'flex';
    document.body.style.overflow = 'hidden'; // Prevent background scroll
}
function closeVentureModal() {
    document.getElementById('ventureModal').style.display = 'none';
    document.body.style.overflow = '';
}
```

The modal form has two sections:

- **Client Identity** — Legal Name, Contact
- **Project Specification** — Project Name, Location, Type, Budget, Start Date, Scope description

jQuery validation on the modal form:

```
$('#ventureForm').validate({
    rules: {
        client_name: "required",
        project_name: "required",
        location: "required",
        project_type: "required"
    }
});
```

Projects present:

Project	Location	Status	Progress
Rajkot Smart City Plaza	Rajkot Infrastructure District	Construction Phase	72%
Matru Ashish	Khambhalia Heights	Pre-Approval	15%
Morbi Ceramic Hub	Morvi District	Industrial	88%
Shanti Sadan	Rajkot	Active	Various

4.5 `payment_gateway.php`

Purpose: Financial oversight page showing all transactions — money collected from clients and payouts to workers/suppliers.

How data is structured (PHP array):

```php
$stats = [
    'total_revenue'       => '₹ 1.24 Cr',
    'pending_collections' => '₹ 18.5 L',
    'workforce_payouts'   => '₹ 4.2 L',
    'active_contracts'    => '14'
];

$transactions = [
    ['id' => 'TXN-9021', 'project' => 'RMC Smart City Plaza',
     'party' => 'Municipal Corporation', 'amount' => '₹ 25,00,000',
     'type' => 'Collection', 'status' => 'synchronized', 'date' => 'Feb 12, 2026'],
    // ... more rows
];
```

Four stat cards:

Card	Color border	Value
Total Revenue	Gray	₹ 1.24 Cr
Pending Collections	Rajkot Rust (red)	₹ 18.5 L
Scheduled Payouts	Amber	₹ 4.2 L
Active Contracts	Slate	14

Transaction table rendering with PHP loops:

```
<?php foreach($transactions as $t): ?>
<tr>
    <td><?php echo $t['id']; ?></td>
    <td><?php echo $t['project']; ?></td>
    <td>
        <!-- Amount shown green for Collection, dark for Payout -->
        <span class="<?php echo $t['type'] === 'Collection' ? 'text-approval-green' : 'text-fou
            <?php echo $t['type'] === 'Payout' ? '-' : '+'; ?><?php echo $t['amount']; ?>
        </span>
    </td>
    <td>
        <!-- Status badge: synchronized / pending / failed -->
        <?php if ($t['status'] === 'synchronized'): ?>
            <span class="text-approval-green">● Synchronized</span>
        <?php elseif ($t['status'] === 'pending'): ?>
            <span class="text-pending-amber animate-pulse">● Awaiting Sync</span>
        <?php else: ?>
            <span class="text-red-600">● Reconcile Fail</span>
        <?php endif; ?>
    </td>
</tr>
<?php endforeach; ?>
```

Status indicators:

- **Synchronized** (green, pulsing dot) = Transaction completed
- **Awaiting Sync** (amber, animated) = Payment pending
- **Reconcile Fail** (red) = Payment failed

Transaction types:

- **Collection** = Money coming IN from a client (shown with `+`, green)
- **Payout** = Money going OUT to worker/supplier (shown with `-`, dark)

Action buttons per row:

- Receipt icon → View invoice/receipt
- History icon → Audit trail

4.6 `leave_management.php`

Purpose: Allows admin to approve or reject employee leave requests.

No database in this version — All leave data is hardcoded HTML for the CIE demo.

Three summary stat cards at top:

- **Pending** — 12 requests (amber border)
- **Approved** — 24 this month (green border)
- **On Leave Today** — 5 people (rust border)

Leave table columns:
Employee | Type | Dates | Reason | Status | Actions

Leave approval/rejection logic (JavaScript only):

```javascript
function handleLeaveAction(employeeName, action, btn) {
    const verb = action === 'approved' ? 'Authorize' : 'Decline';

    if (confirm(`Are you sure you want to ${verb} the leave for ${employeeName}?`)) {

        // 1. Update the status cell in the same table row
        const row = btn.closest('tr');
        const statusCell = row.querySelector('[data-label="Status"]');
        statusCell.innerHTML = `<span class="bg-green-50 text-green-700">● APPROVED</span>`;

        // 2. Show a toast notification (floating message)
        const notification = document.createElement('div');
        notification.className = 'fixed bottom-8 right-8 bg-green-600 text-white px-8 py-4';
        notification.innerHTML = `<b>${employeeName}</b> is ${action}.`;
        document.body.appendChild(notification);

        // 3. Auto-remove notification after 3 seconds
        setTimeout(() => notification.remove(), 3000);

        // 4. Replace action buttons with "Processed" text
        const wrapper = row.querySelector('.actions-wrapper');
        wrapper.innerHTML = `<span>Processed</span>`;
    }
}
```

Key concepts demonstrated:

- `btn.closest('tr')` → traverses up the DOM tree to find the parent row
- `document.createElement()` → dynamically creates a new HTML element
- `setTimeout(() => ..., 3000)` → runs code after 3 second delay
- `confirm()` → native browser confirmation dialog

Leave statuses in the table:

- **Pending** (amber) — Not yet reviewed
- **Approved** (green) — Admin approved
- **Rejected** (red) — Admin declined

4.7 `file_viewer.php`

Purpose: View project blueprints/PDF files inside the browser without downloading.

URL parameter:

```
$file = $_GET['file'] ?? null;
$fileName = $file ? basename($file) : 'Blueprint_A1_01.pdf';
// basename() extracts just the filename, preventing directory traversal attacks
```

Two-panel layout:

- **Left sidebar** — File Details (project name, version, status) + Revision History
- **Right main area** — PDF canvas + toolbar

Toolbar controls (JavaScript):

```
let currentZoom = 100;

function updateZoom(delta) {
    currentZoom = Math.max(50, Math.min(200, currentZoom + delta));
    document.getElementById('zoom-level').textContent = currentZoom + '%';
    document.getElementById('pdf-container').style.transform = `scale(${currentZoom / 100})`;
}
```

- Zoom In: `+10%` → max 200%
- Zoom Out: `-10%` → min 50%
- Print: `window.print()`

- Download: `handleDownload()`

Mobile sidebar toggle:

```
function toggleSidebar() {
    const sidebar = document.getElementById('fileSidebar');
    sidebar.classList.toggle('hidden');
}
```

Revision History displayed:

Version	Date	Note
v2.4 (Current)	Feb 15	Minor structural edits
v2.3	Feb 02	Initial submission
v2.2	Jan 20	Schematic approval
v2.1	Jan 05	Conceptual draft

Blueprint "watermark" effect:

```
<div class="flex-grow flex items-center justify-center">
    <!-- Giant watermark text -->
    <p class="text-9xl rotate-[-45deg] border-4 border-gray-100 opacity-50">Ripal Design</p>
</div>
```

This gives the placeholder a realistic blueprint/drawing feel.

5. Design System

Color Palette

Color Name	Hex	Used For
`rajkot-rust`	`#94180C`	Primary accent, CTA buttons, error states, brand
`foundation-grey`	`#2D2D2D`	Headers, dark backgrounds, body text

Color Name	Hex	Used For
`canvas-white`	`#F9FAFB`	Page background (off-white)
`approval-green`	`#15803D`	Success, approved status, positive values
`pending-amber`	`#B45309`	Warning, pending status, scheduled items
`slate-accent`	`#334155`	Workers, secondary information

Typography

Font	Style	Used For
Inter	Sans-serif	Body text, labels, small text, buttons
Playfair Display	Serif	Page headings (H1, H2)
Cormorant Garamond	Serif	Alternate heading style

CSS Framework — Tailwind CSS

All styling is done using **Tailwind utility classes** loaded from CDN.

Common patterns used:

```
bg-foundation-grey text-white     → Dark header
bg-rajkot-rust hover:bg-red-700   → Primary button
border-b-2 border-rajkot-rust     → Rust bottom border accent
shadow-premium                    → Custom soft shadow
font-serif font-bold              → Playfair Display heading
text-[10px] uppercase tracking-widest → Labels/captions
```

Responsive Design

Pages use Tailwind breakpoint prefixes:

- `md:` → applies on medium screens (768px+)
- `lg:` → applies on large screens (1024px+)
- `sm:` → applies on small screens (640px+)

Example: `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3` → 1 column on mobile, 2 on tablet, 3 on desktop.

6. Form Validation

Two levels of validation are used: **HTML5** and **PHP server-side**, with some forms also using **jQuery Validation Plugin**.

Level 1 — HTML5 Built-in Validation

```
<input type="email" required>            <!-- Must be valid email format -->
<input type="password" minlength="8">   <!-- Minimum 8 characters -->
<input type="text" required>            <!-- Cannot be empty -->
<select required>                        <!-- Must select an option -->
```

These show browser-native error bubbles. They run **before** the form is submitted.

Level 2 — PHP Server-side Validation (add_user.php)

```
// 1. Check if any field is empty
if (empty($firstName) || empty($lastName) || empty($email) || empty($password)) {
    $error = 'All fields are required.';
}
// 2. Validate email format
elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $error = 'Please enter a valid email address.';
}
// 3. Check for duplicate email in database
else {
    $stmt = $db->prepare("SELECT id FROM users WHERE username = ?");
    $stmt->execute([$email]);
    if ($stmt->fetch()) {
        $error = 'A user with this email already exists.';
    }
}
```

Error messages displayed with:

```php
<?php if ($error): ?>
    <div class="bg-red-50 border-l-4 border-rajkot-rust" role="alert">
        <?php echo htmlspecialchars($error); ?>  <!-- htmlspecialchars prevents XSS -->
    </div>
<?php endif; ?>
```

Level 3 — jQuery Validation Plugin (provision_temp_user.php, project_management.php)

```jquery validation
$("#tempProvisionForm").validate({
    rules: {
        firstName: "required",
        email: { required: true, email: true },
        role: "required"
    },
    messages: {
        firstName: "First name is mandatory",
        email: "Valid email address required",
        role: "Please select a system role"
    }
});
```

This runs **on submit** and shows styled error messages directly under each field without page reload.

Why `htmlspecialchars()` on output?

- Prevents **XSS (Cross-Site Scripting)** attacks
- If a user types `<script>alert(1)</script>` in their name, `htmlspecialchars()` converts `<` to `<` so the browser displays it as text, not code
- **Always use on any user-supplied data you print to the page**

7. Authentication & Role Guard

How It Works — Step by Step

Step 1: User logs in

```
public/login.php → validates credentials → sets $_SESSION['user'] = [
    'id'    => 5,
    'email' => 'admin@ripaldesign.in',
    'role'  => 'admin',
    'name'  => 'Ashish Vinchhi'
]
```

Step 2: Admin tries to visit an admin page

```php
// admin/user_management.php — first two lines:
require_once __DIR__ . '/../includes/init.php';
require_role('admin');
```

Step 3: `require_role('admin')` checks the session

```php
function require_role($role) {
    require_login();          // First: are they logged in at all?
    if (!has_role($role)) {   // Second: do they have the right role?
        header('Location: /public/index.php');
        exit;
    }
}
```

Step 4: If a worker tries to visit an admin page

- `has_role('admin')` checks `$_SESSION['user']['role']` — it's `'worker'`, not `'admin'`
- Returns `false` → redirect to homepage
- The worker never sees the page

Step 5: If nobody is logged in

- `$_SESSION['user']` is empty
- `require_login()` redirects to `public/login.php`
- `exit` stops any further PHP execution

8. Database Connection Pattern

Prepared Statements — Why They Matter

UNSAFE (never do this):

```
// SQL Injection vulnerability!
$email = $_POST['email'];
$result = $db->query("SELECT * FROM users WHERE username = '$email'");
// Attack: person types: admin@x.com' OR '1'='1  → logs in as anyone
```

SAFE (what we use):

```
$stmt = $db->prepare("SELECT id FROM users WHERE username = ?");
$stmt->execute([$email]);
$user = $stmt->fetch();
```

The `?` is a **placeholder**. The database treats the user input as pure data (never as SQL code), so injection is impossible.

Named placeholders (alternative):

```
$stmt = $db->prepare("INSERT INTO users (username, password_hash, role) VALUES (:email, :hash,
$stmt->execute([':email' => $email, ':hash' => $hash, ':role' => $role]);
```

Fetch Methods

```
$stmt->fetch()      // Returns one row as array, false if none
$stmt->fetchAll()   // Returns all rows as array of arrays
$stmt->fetchColumn()// Returns single value from first column
```

9. Common Questions & Answers

Q: What is the purpose of your admin panel?

A: The admin panel is the control centre for Ripal Design — an architecture firm. It allows the principal

admin to manage all users (clients, workers, employees), track projects with their progress, monitor financial transactions, approve or reject employee leave requests, and view project blueprint files.

Q: How do you prevent unauthorised users from accessing admin pages?

A: Every admin page starts with two lines: `require_once '../includes/init.php'` and `require_role('admin')`. The `require_role()` function in `auth.php` checks the PHP session — if the user is not logged in or their role is not 'admin', they are immediately redirected and the page execution stops with `exit`.

Q: How do you validate forms?

A: I use three layers. First, HTML5 attributes like `required`, `type="email"`, and `minlength` give instant browser feedback. Second, PHP server-side validation checks for empty fields, validates email format using `filter_var()`, and checks for duplicate users in the database. Third, on complex forms like user provisioning, I use the jQuery Validation plugin which shows inline error messages under each field without page reload.

Q: How are passwords stored?

A: I never store plain text passwords. I use PHP's `password_hash($password, PASSWORD_DEFAULT)` which uses the bcrypt algorithm to create a one-way hash. When verifying on login, `password_verify($inputPassword, $storedHash)` is used to compare. The actual password can never be recovered from the hash.

Q: What are prepared statements and why do you use them?

A: Prepared statements separate SQL code from user data using placeholders (`?`). When we call `$stmt->execute([$email])`, the database treats the value as data, not SQL code. This completely prevents SQL injection attacks where a malicious user might try to input SQL commands to bypass authentication or delete data.

Q: What is `htmlspecialchars()` and where do you use it?

A: `htmlspecialchars()` converts special HTML characters like `<`, `>`, `"` into their safe HTML entities

(`<` , `>` , `"`). I use it every time I output user-supplied data to the page. This prevents XSS (Cross-Site Scripting) attacks where someone could inject JavaScript into their username or email to steal other users' session cookies.

Q: How is your website mobile responsive?

A: I use Tailwind CSS with responsive breakpoint prefixes. For example: `grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3` makes a 1-column grid on phones, 2-column on tablets, 3-column on desktops. Tables convert to card stacks on mobile using CSS with `data-label` attributes so column headers still show. The navigation uses a hamburger button that opens a full-screen sidebar panel.

Q: What is PDO and why use it over `mysqli` ?

A: PDO (PHP Data Objects) is a database-abstraction layer that supports multiple database types (MySQL, PostgreSQL, SQLite, etc.) with the same API. `mysqli` only works with MySQL. PDO also has cleaner object-oriented syntax and better support for prepared statements. We set `ATTR_ERRMODE_EXCEPTION` so database errors throw PHP exceptions that we can catch and handle gracefully.

Q: How does the page work without a database?

A: I implemented a fallback/demo mode. In `db.php` , if the PDO connection fails, `$pdo` is set to `null` . Each page checks `if (isset($pdo) && $pdo instanceof PDO)` before querying. If no database, it uses hardcoded static data arrays. This means the full UI can be demonstrated even in environments without a running MySQL server — like during a CIE presentation.

Q: Explain the "Provision Temporary Identity" page.

A: This page lets an admin create accounts for workers or employees without requiring them to self-register. The admin fills in the name, email, and role. For workers and employees, a second dropdown dynamically appears (using jQuery) with job-specific designations like "Electrical Specialist" or "Principal Structural Engineer". On submission, PHP generates a random 8-character temporary password using `bin2hex(random_bytes(4))` , which the admin can then share with the person. This is more controlled than open registration.

Q: How does the Leave Management page work?

A: Leave requests are displayed in a table showing the employee name, leave type, dates, and reason. When the admin clicks Approve or Reject, a JavaScript `confirm()` dialog asks for confirmation. On confirmation, the JS updates the status cell in that row using DOM manipulation, replaces the action buttons with "Processed" text, and shows a toast notification that auto-disappears after 3 seconds using `setTimeout`. This currently runs without a database — in CIE-2, it would be connected to a database with UPDATE queries.

Q: What are Lucide Icons?

A: Lucide is a modern open-source icon library. We include it via CDN: `<script src="https://unpkg.com/lucide@latest"></script>`. Icons are used as `<i data-lucide="trash-2"></i>` attributes. After the page loads, `lucide.createIcons()` replaces those elements with inline SVG icons. They're sharp at any size and consistent with the modern design.

Q: Explain the project card grid and filter.

A: Each project card has `data-region="Rajkot"` and `data-status="Construction Ongoing"` attributes. The JavaScript `filterRegion()` function loops through all `.project-card` elements and sets `display: none` on cards whose `dataset.region` doesn't match the selected filter. This is pure front-end filtering — no page reload, no database. In CIE-2 this would be replaced with server-side filtering via GET parameters.

10. Quick-Reference Code Snippets

Protect a page (require admin role)

```php
require_once __DIR__ . '/../includes/init.php';
require_role('admin');
```

Handle a POST form safely

```php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $name = trim($_POST['name'] ?? '');
    if (empty($name)) { $error = 'Name is required'; }
    // ... process if no error
}
```

Run a SELECT query

```php
$db = get_db();
$stmt = $db->prepare("SELECT * FROM users WHERE role = ?");
$stmt->execute(['admin']);
$users = $stmt->fetchAll();
```

Run an INSERT query

```php
$stmt = $db->prepare("INSERT INTO users (username, password_hash, role) VALUES (?, ?, ?)");
$stmt->execute([$email, password_hash($password, PASSWORD_DEFAULT), $role]);
```

Show error message from PHP

```php
<?php if ($error): ?>
    <div class="alert"><?php echo htmlspecialchars($error); ?></div>
<?php endif; ?>
```

jQuery Validation setup

```jquery
$("#myForm").validate({
    rules: { fieldName: "required" },
    messages: { fieldName: "This field is required" }
});
```

Dynamic show/hide with jQuery

```
$('#roleSelect').on('change', function() {
    if ($(this).val() === 'worker') {
        $('#extraDiv').removeClass('hidden');
    } else {
        $('#extraDiv').addClass('hidden');
    }
});
```

Toast notification (no library)

```
const toast = document.createElement('div');
toast.className = 'fixed bottom-8 right-8 bg-green-600 text-white px-6 py-3';
toast.textContent = 'Action completed!';
document.body.appendChild(toast);
setTimeout(() => toast.remove(), 3000);
```

Redirect after delay (PHP)

```
header("Refresh: 2; url=user_management.php");
// Or immediate redirect:
header("Location: user_management.php");
exit;
```

Study tip: For the UI tweak section of CIE-1, know where the color variables are (`rajkot-rust` , `foundation-grey`) and how to change them in Tailwind classes. Know where the heading text is in each page's `<header>` section. Know how to add/remove a field from a form.